

## REACTJS (ASSIGNMENT 4)

### Aim:

To create a React application that fetches post data from a REST API and displays it on the webpage using React components.

### Steps Performed

1. Created a new React application using:  
npx create-react-app scorecalculatorapp

```
PS C:\Users\Lenovo> npx create-react-app blogapp
Success! Created blogapp at C:\Users\Lenovo\blogapp
Inside that directory, you can run several commands:

  npm start
    Starts the development server.

  npm run build
    Bundles the app into static files for production.

  npm test
    Starts the test runner.

  npm run eject
    Removes this tool and copies build dependencies, configuration files
    and scripts into the app directory. If you do this, you can't go back!

We suggest that you begin by typing:

  cd blogapp
  npm start

Happy hacking!
PS C:\Users\Lenovo>
```

2. Opened the project in Visual Studio Code.

```
▼ BLOGAPP
  > node_modules
  > public
  ▼ src
    # App.css
    JS App.js
    JS App.test.js
    # index.css
    JS index.js
    logo.svg
    JS Post.js
    JS Posts.js
    JS reportWebVitals.js
    JS setupTests.js
    .gitignore
    {} package-lock.json
    {} package.json
    ⓘ README.md
```

3. Created a new component named **Posts.js** inside the **src** folder.
4. Used Axios to fetch data from the following REST API:  
`https://jsonplaceholder.typicode.com/posts`
6. Displayed the post title and body using React.
7. Imported the Posts component into App.js.
8. Executed the application using:  
`npm start`
9. Verified the output in the browser.

#### **Post.js:**

```
class Post {  
  constructor(id, title, body) {  
    this.id = id;  
    this.title = title;  
    this.body = body;  
  }  
}  
export default Post;
```

#### **Posts.js:**

```
import React, { Component } from 'react';  
import Post from './Post';  
class Posts extends Component {  
  constructor(props) {  
    super(props);  
    this.state = {  
      posts: []  
    };  
  }  
  loadPosts() {  
    fetch("https://jsonplaceholder.typicode.com/posts")  
      .then(response => response.json())  
      .then(data => {  
        let posts = data.map(item =>  
          new Post(item.id, item.title, item.body)  
        );  
        this.setState({  
          posts: posts  
        });  
      });  
  }  
  componentDidMount() {  
    this.loadPosts();  
  }  
  componentDidCatch(error, info) {  
    alert(error);  
  }  
}
```

```

render() {
  return (
    <div>
      <h1>Posts</h1>
      {
        this.state.posts.map(post => (
          <div key={post.id}>
            <h3>{post.title}</h3>
            <p>{post.body}</p>
            <hr />
          </div>
        ))
      }
    </div>
  );
}
}
export default Posts;

```

### App.js:

```

import './App.css';
import Posts from './Posts';

```

```

function App() {

  return (
    <div className="App">
      <Posts />
    </div>
  );
}
export default App;

```

### Output:

```

PS C:\Users\Lenovo\blogapp> npm start

> blogapp@0.1.0 start
> react-scripts start

(node:17476) [DEP_WEBPACK_DEV_SERVER_ON_AFTER_SETUP_MIDDLEWARE] DeprecationWarning: 'onAfterSetupMiddleware' option is
deprecated. Please use the 'setupMiddlewares' option.
(Use `node --trace-deprecation ...` to show where the warning was created)
(node:17476) [DEP_WEBPACK_DEV_SERVER_ON_BEFORE_SETUP_MIDDLEWARE] DeprecationWarning: 'onBeforeSetupMiddleware' option
is deprecated. Please use the 'setupMiddlewares' option.
Starting the development server...
Compiled successfully!

You can now view blogapp in the browser.

  Local:            http://localhost:3000
  On Your Network:  http://192.168.0.103:3000

Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
PS C:\Users\Lenovo\blogapp>

```